

E2: Text Processing and Text Mining

by Ludovico Grabau

Part 1: Text Processing

1. Regular Expressions for ICD-10 Codes (subsections 2.7.1-2.7.3)

What is regex?

Regular expressions (regex), as introduced in Jurafsky (subsections 2.7.1–2.7.3), are a powerful way to describe textual patterns. By combining **character classes**, **repetition counters**, and **anchors/word boundaries**, regex allows us to precisely define the “shape” or “form” of a string.

ICD-10 diagnosis codes follow a relatively consistent structure which a regex can capture. For example:

- 1 uppercase letter (**A–Z, including U**),
- 2 digits,
- optional “.” + **1–4 alphanumeric** characters.

In addition, Word boundaries/anchors ensure matches are standalone tokens, not just part of a larger string. In practice, such patterns can scan clinical narratives and extract likely ICD-10 codes quickly (e.g, **E11.9**, **C50.11**, or **Z99.2**). This makes regex a convenient, fast, resource-light first-pass.

Example Pattern:

```
import re
# Letter A-Z + 2 digits + optional '.' + 1-4 alphanumerics
# ICD-10(-CM) allows 3-7 characters, excluding dot as formatting
# (1st alpha, 2-3 digits, then 0-4 alphanumerics)
# Ignore case to match lowercase letters from typos (e.g., e11.9)
icd10_pattern = re.compile(r"\b[A-Z][0-9]{2}(?:\.[A-Z0-9]{1,4})?\b", flags=re.IGNORECASE)
matches = re.findall(icd10_pattern, some_text)
```

Optional (more robust boundaries if codes are glued to letters/underscores):

```
icd10_pattern_strict = re.compile(
    r"(?<![A-Za-z0-9])[A-Za-z][0-9]{2}(?:\.[A-Za-z0-9]{1,4})?(?![A-Za-z0-9])",
    flags=re.IGNORECASE
)
```

Regex Terminology:

- [A-Z] → character class
- {2} and {1,4} → repetition counters
- ^, \$, \b → anchors/word boundaries
- (?: ...) → non-capturing group, simplifying findall usage

Note: Depending on the type **WHO ICD-10** vs. **ICD-10-CM** (US extension), the code needs to be adjusted for validation accordingly, since allowable characters and lengths after the dot differ by code set. Historically **U** was reserved, but modern practice (e.g., **U07.1** for COVID-19) uses **U** codes.

Limitations of Regex

While useful, regex alone has several drawbacks and disadvantages for ICD-10 extraction:

- **False positives:** Matches well-formed but nonexistent codes (e.g., **A00.ZZ**). Regex cannot distinguish valid ICD-10 entries from invalid ones, so there is no verification of their validity and no guarantee of their clinical correctness.
- **False negatives:** Misses noisy or non-standard variants with spaces, lowercase letters, missing dots, or other typos, unless rules are broadened. Regex patterns are weak in noisy, real-world text.
- **No semantics or context:** Cannot determine whether a code refers to the patient, is negated (“no E11.9”), historical, or hypothetical, serving merely as an example.
- **Maintenance burden:** Rules may need updates for edge cases, evolving standards, or different code sets (e.g., ICD-10-PCS).

Best Practice

Use regex as a **quick first filter**, first extract and **normalize** candidates (uppercase + add dot after 3rd character if missing), then **validate** with an official ICD-10/ICD-10-CM list to remove invalid codes, and/or finally apply **NLP** (e.g., clinical NER) to determine relevance and to handle context, like negation, experimenter, and temporality.

2. spaCy Exercises

Exercise 1: Named Entity Recognition (NER) with spaCy

Input sentence: “*The patient was prescribed 5 mg of Prednisone in Zurich.*”

spaCy “en_core_web_md” output (detected entities):

- “5 mg” → **QUANTITY** (*reasonable but not clinical*)
- “Prednisone” → **GPE** (*misclassified as geopolitical entity*)
- “Zurich” → **GPE** (*correct*)

Missed important terms (incorrect entities):

- **Prednisone** should have been identified as a **DRUG**.
- **5 mg** should ideally be labeled as **DOSAGE**, not just a generic **QUANTITY**.
- Clinical categories like **DRUG**, **DOSAGE**, **ROUTE**, **FREQUENCY** are missing entirely.

Interpretation:

This example shows both the strengths and limitations of general-purpose NER models. spaCy can detect surface-level entities like quantities and locations but misclassifies clinical terms. For example: labeling *Prednisone* as a place and missing medication attributes. This reflects a broader issue, general English NER covers broad categories but lacks clinical knowledge. While they capture basic cues (numbers, units, names), reliable extraction of medical concepts requires biomedical models with explicit categories such as **drug**, **dosage**, **route of administration**, and **frequency**.

Exercise 2: Custom Rule-Based Matcher

What the code is doing:

This code uses **EntityRuler**, a rule-based spaCy component, to capture clinical entities that the general model misses. Custom rules target:

- **DOSAGE**: number + unit (e.g., *5 mg*, *10 ml*)
- **MEDICATION/DRUG**: suffix-based regex rules (e.g., *-ine*, *-ol*, *-pril*, *-sartan*, *-mab*) together with a small drug lexicon (*prednisone*, *ibuprofen*)
- **ROUTE**: abbreviations for administration methods (*PO*, *IV*, *IM*, *SC*)
- **FREQUENCY**: dosing expressions (*once daily*, *twice daily*, *bid*, *weekly*)

The EntityRuler runs before the **ner** component, so rules apply first, while the statistical model adds broader predictions. Also, it retains the base model's geopolitical entity recognition (e.g., *Zurich*). This hybrid pipeline successfully marks clinical entities with high precision, while still retaining diverse entity recognition from spaCy's general NER.

Problems with coding like this:

While rules are effective for predictable structures, they come with important limitations:

- **Limited coverage**: new drugs or phrasing must be added manually.
- **False positives**: regex rules can over-label (e.g., any word ending in *-ine*).
- **Context blindness**: rules do not handle meaning, so *"not taking ibuprofen"* would still be labeled as a drug.
- **Overlap/conflicts**: multiple rules may clash, and their order affects outcomes.
- **No generalization**: the system cannot adapt or learn beyond what is explicitly encoded.

Interpretation:

Rule-based matchers excel at **structured and repeatable patterns** (e.g., dosages, drugs, routes, frequencies) and offer precision but lack flexibility when handling messy clinical text alone. They work best combined with **statistical models**, not replaced by them.

Exercise 3: Train a tiny custom NER model in spaCy

Pipeline Setup:

A blank English pipeline (`spacy.blank("en")`) is created with only the **ner** component. This means predictions rely solely on token and context embeddings, without POS tags, parsing, or lemmatization. Training is carried out on a few annotated examples covering four baseline labels:

- **DOSAGE** (e.g., *10 mg*)
- **DRUG** (e.g., *morphine*)
- **BODY_PART** (e.g., *arm*)
- **MEASUREMENT** (e.g., *2 cm*)

The model is trained for 20 epochs, with data shuffled each epoch, updated using stochastic gradient descent, and loss values tracked. Evaluation is performed on the classical probe sentence: *"The nurse gave 10 mg morphine for arm pain."*

Predicted Entities:

Baseline (4 labels):

Output: [(“10 mg”, DOSAGE), (“morphine”, DRUG)]

With the tiny dataset, training loss quickly falls to zero as the model memorizes patterns like `NUM + unit → DOSAGE`. It identifies dosage and drug but misses *arm* as a `BODY_PART`, showing it learns template-like rules and frequent tokens rather than generalizing to unseen cases.

Extended (10 labels, 1–2 examples each → Please refer to Spacy-exercises to see them):

Output: [(“10 mg”, DOSAGE), (“morphine”, DRUG), (“arm”, BODY_PART)]

Adding six new labels with only one or two examples each raises training loss at first due to sparse data, but it soon converges near zero as the model memorizes the tiny dataset. A small gain appears, *arm* is now recognized as a `BODY_PART`, but the new labels rarely activate beyond near-identical cases. For instance, a single *panel* example as `LAB_TEST` can cause the model to over-tag all mentions of *panel* the same way.

Qualitative Evaluation

The model handles regular patterns well (e.g., `NUM + unit → DOSAGE`), easily memorizes drug names, and shows slight recall improvements with added labels. However, it overfits due to the small dataset, struggles with inconsistent spans, confuses similar labels (e.g., dosage vs. measurement), rarely applies sparse labels unless inputs match training data, and produces false positives from superficial matches like tagging any *panel* as `LAB_TEST`.

Interpretation

Training a tiny spaCy NER model with limited data highlights clear limits: it memorizes quickly, works only on training-like inputs, and fails on others. Adding new labels raises early loss and slightly improves recall, but most remain weak. The model handles simple patterns and frequent tokens but struggles with rare or complex labels, causing overfitting, inconsistent spans, and false positives. Better performance requires consistent spans, more varied examples, pretrained biomedical embeddings (e.g., SciSpaCy), and rule-based matchers for regular entities like dosages.

3. Discuss briefly why and when to use a stemmer, when one can have lemmatizer?

Stemming and lemmatization both reduce words to a base form, but with different trade-offs.

Stemming chops words to crude roots by applying simple rules (e.g., *running* → *run*, *studies* → *studi*). It is very fast, requires no external resources, and works well in information retrieval (IR) or search where recall matters more than exact morphology (e.g., matching “connect,” “connected,” and “connection”). However, it often leads to over-stemming (*universe* ↔ *university*) and under-stemming, produces stems that are not always real words, and makes language- or domain-sensitive errors. A stemmer is best used when speed and broad recall are more important than linguistic precision, such as in bag-of-words IR systems, large-scale deduplication, or quick-and-dirty text normalization.

Lemmatization, by contrast, maps words to linguistically correct base forms (lemmas) using dictionaries and part-of-speech/morphological analysis (e.g., *better* → *good*, *saw* → *see* (verb), *saws* → *saw* (noun)). It provides higher precision and interpretability, making it more suitable for downstream NLP tasks like parsing, named entity recognition, semantic similarity, or topic modeling with interpretable features. The trade-offs are that it is slower, depends on linguistic resources or models, and may struggle with noisy text or low-resource languages. Lemmatizers are preferred when correctness matters, for example, in information extraction, clinical text mining, or rule-based systems where canonical word forms (e.g., *diagnoses/diagnosed/diagnosis* → *diagnosis*) lead to cleaner and more consistent features than crude stems.

Part 2: Text Mining

1. Solve the exercise on slide 10 of week 3.

```
import numpy as np

# Vocabulary order (from the slide)
vocab = ["fever", "cough", "shortness of breath", "sore throat", "fatigue"]

# Symptom frequency vectors
# Report 1: "fever, cough, shortness of breath"
# Report 2: "fever, sore throat, fatigue"
r1 = np.array([1, 1, 1, 0, 0], dtype=float)
r2 = np.array([1, 0, 0, 1, 1], dtype=float)

def cosine_similarity(a, b):
    return float(a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

def normalize(v):
    n = np.linalg.norm(v)
    return v / n if n != 0 else v

# Cosine similarity
cos_sim = cosine_similarity(r1, r2)

# Euclidean distance between unit-normalized vectors
r1_u, r2_u = normalize(r1), normalize(r2)
euclid_dist_unit = float(np.linalg.norm(r1_u - r2_u))

# Theoretical relationship for unit vectors:  $d = \sqrt{2 * (1 - \cos)}$ 
relationship = float(np.sqrt(2 * (1 - cos_sim)))

print("Vocabulary order:", vocab)
print("Report 1 vector:", r1.tolist())
print("Report 2 vector:", r2.tolist())
print(f"1. Cosine similarity = {cos_sim:.6f} (exactly 1/3 ~= 0.333333)")
print(f"2. Euclidean distance between normalized vectors = {euclid_dist_unit:.6f}")
print(f"    and  $d = \sqrt{2(1 - \cos)}$  = {relationship:.6f}")
```

(1) Cosine Similarity

Symptom	Frequency in R1	Frequency in R2
fever	1	1
cough	1	0
shortness of breath	1	0
sore throat	0	1
fatigue	0	1

Using the symptom-frequency vectors:

$$\text{Cosine Similarity} = \frac{1 \cdot 1 + 1 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 1}{\sqrt{1^2 + 1^2 + 1^2 + 0^2 + 0^2} \sqrt{1^2 + 0^2 + 0^2 + 1^2 + 1^2}} = \frac{1}{\sqrt{3}\sqrt{3}} = \frac{1}{3} \approx 0.3333.$$

(2) Relation to Euclidean Distance

After L2-normalization of both vectors, the Euclidean distance is,

$$d = |\hat{r}_1 - \hat{r}_2| = \sqrt{2(1 - \cos \theta)} = \sqrt{2(1 - \frac{1}{3})} = \sqrt{\frac{4}{3}} \approx 1.1547.$$

Cosine similarity and Euclidean distance are not numerically equal, but for unit vectors they are a simple monotonic transform of each other via, $d = \sqrt{2(1 - \cos \theta)}$, so they induce the same ranking of similarity.

(3) Benefit of Using SNOMED CT

Yes, mapping terms to SNOMED CT concepts would improve the results:

- Handles synonyms and variants which reduces vocabulary mismatch (e.g., “*shortness of breath*” → “*dyspnea*”).
- Exploits hierarchical relations (e.g., grouping under “respiratory symptoms”).
- Captures semantic similarity beyond exact word overlap improving the overall similarity measure between reports.

This provides a more medically meaningful comparison than raw symptom frequency vectors.

2. Why is padding the text not useful here?

Padding is needed in **sequence models** (e.g., RNNs, LSTMs, CNNs, Transformers) to align input lengths. In this program (notebook), however, neither pipeline relies on sequential processing. Instead, the model used is a feed-forward neural network, for which padding provides no significant benefit.

Naive Bayes + TF-IDF

`TfidfVectorizer` turns each document into a fixed-size **bag-of-words** vector of vocabulary length. Word order is ignored, so reviews of any length map to the same dimensionality. Adding padding would either be ignored or act as a meaningless feature.

Keras Embedding + GlobalAveragePooling1D

```
x = tf.keras.layers.Embedding(vocab_size, 128, mask_zero=True)(x)
x = tf.keras.layers.GlobalAveragePooling1D()(x)
```

With `mask_zero=True`, the embedding layer ignores padding tokens, and `GlobalAveragePooling1D` averages only the real tokens. Variable-length reviews are thus handled naturally without padding.

Caveat: without masking (`mask_zero=False`) or mask-aware pooling, padding could distort results. In that case, one would need masking or ragged tensors.

Interpretation

- **Naive Bayes + TF-IDF**: already fixed-length vectors → no padding needed.
- **Embedding + Pooling**: masking handles variable lengths.

Therefore, padding would only be required in true sequence models. Also, when using `mask_zero=True`, we must ensure that index 0 is reserved for padding, otherwise genuine tokens mapped to 0 will be masked.